AWS Classroom Deployment Guide

AWS CLI • CloudShell • Docker Desktop • Amazon ECR • Amazon ECS Fargate

Student lab | Example application: Hello World / PaySprint Wealth Platform

Goal: Authenticate to the correct AWS account, push a Docker image to ECR, and run it as an ECS Fargate service. All account IDs, URLs, roles, and image tags shown here are examples.

1. Understand the deployment flow

Docker Desktop (Mac) → Amazon ECR (image registry) → Amazon ECS Fargate (running task/container). AWS CloudShell is a browser-based AWS terminal used to manage AWS resources; it does not automatically see local Docker Desktop images. ECR stores images; ECS runs containers.

2. Choose the AWS region

Use one region consistently for ECR, ECS, and other regional resources. This guide uses US East (Ohio), us-east-2. IAM Identity Center may use a different home region.

Region	Code
US East (N. Virginia)	us-east-1
US East (Ohio)	us-east-2
US West (Oregon)	us-west-2
Europe (Ireland)	eu-west-1
Europe (Frankfurt)	eu-central-1
Asia Pacific (Sydney)	ap-southeast-2

List regions available to your account (subject to permissions):

```
aws account list-regions --region-opt-status-contains ENABLED_BY_DEFAULT ENABLED --query "Regions[*].RegionName" --output table
```

3. AWS CLI on a student computer (optional)

AWS CLI v2 is installed separately from Docker Desktop. Windows: https://awscli.amazonaws.com/AWSCLIV2.msi ; macOS: https://awscli.amazonaws.com/AWSCLIV2.pkg (or brew install awscli); Linux: use the official AWS CLI v2 Linux installation instructions.

```
aws --version
aws configure list-profiles
aws sts get-caller-identity
```

IAM Identity Center (SSO) example

An administrator must provide the real IAM Identity Center start URL or issuer URL, SSO region, and AWS account/role assignment. The CLI profile name is a local name you choose; it is not your AWS IAM username.

```
aws configure sso --profile classroom
aws sso login --profile classroom
aws sts get-caller-identity --profile classroom
```

Example SSO configuration (~/.aws/config):

```
[profile classroom]
sso_session = classroom-session
sso_account_id = 123456789012
sso_role_name = StudentDeveloper
region = us-east-2
output = json

[sso-session classroom-session]
sso_start_url = https://d-1234567890.awsapps.com/start
sso_region = us-east-1
sso_registration_scopes = sso:account:access
```

IAM user provided by an organization

A console username/password does not authenticate AWS CLI. If organization policy permits, the IAM user can create a CLI access key in IAM → Security credentials; configure it with aws configure --profile organization. Prefer approved temporary credentials or IAM Identity Center when available. Never create root-user access keys or share secrets.

```
aws configure --profile organization
aws sts get-caller-identity --profile organization
```

Compare Mac and Console identities

In the AWS Console, open the upper-right account/identity menu and note the account ID and IAM user or role. On your Mac, compare Account and Arn returned by STS. Matching account ID alone does not guarantee the same permissions.

```
aws configure list
echo $AWS_PROFILE
aws sts get-caller-identity --profile organization
```

To select a profile for the current macOS terminal session:

```
export AWS_PROFILE=organization
aws sts get-caller-identity
```

4. AWS CloudShell: easiest console-based access

From the AWS Console, select region us-east-2 and open the CloudShell terminal icon. CloudShell uses the console session's authorized AWS identity, subject to IAM permissions. It does not require configuring local Mac access keys.

```
aws sts get-caller-identity
aws configure get region
aws ecr describe-repositories --region us-east-2
```

CloudShell does not automatically contain Docker Desktop images from your Mac. Standard CloudShell environments generally lack a running Docker daemon, so docker load and docker push may not work there. CloudShell is excellent for AWS resource management; push from your Mac with approved credentials, or use AWS CodeBuild/ Jenkins to build and push in AWS.

5. Create an ECR repository

Console: Amazon ECR → Private repositories → Create repository → enter hello-world-svc → Create. Or run in CloudShell:

```
aws ecr create-repository --repository-name hello-world-svc --region us-east-2
aws ecr describe-repositories --repository-name hello-world-svc --region us-east-2
```

The repository URI resembles 123456789012.dkr.ecr.us-east-2.amazonaws.com/hello-world-svc. Replace 123456789012 with your account ID.

6. Push an existing Docker Desktop image to ECR

On your Mac, verify the image and the AWS identity with permission to push:

```
docker images
aws sts get-caller-identity --profile organization
```

Authenticate Docker to ECR (the token expires, so repeat when needed):

```
aws ecr get-login-password --region us-east-2 --profile organization | docker login --username AWS --password-stdin
123456789012.dkr.ecr.us-east-2.amazonaws.com
```

Tag the local image with the full ECR destination, then push:

```
docker tag hello-world-svc:latest 123456789012.dkr.ecr.us-east-2.amazonaws.com/hello-world-svc:v1
docker push 123456789012.dkr.ecr.us-east-2.amazonaws.com/hello-world-svc:v1
```

Verify the image appears in ECR:

```
aws ecr describe-images --repository-name hello-world-svc --region us-east-2 --profile organization
```

If your image was built on Apple Silicon, confirm the image architecture matches the ECS task architecture. For x86_64 Fargate, build with docker buildx build --platform linux/ amd64 ... as appropriate.

Alternatives when Mac CLI credentials are unavailable

Option A: Build from a Git repository using AWS CodeBuild with a service role permitted to push to ECR (Docker builds commonly require privileged mode). Option B: Use the planned Jenkins EC2 server to build and push. CloudShell file upload of a Docker image archive is not, by itself, an ECR push; a suitable container runtime/tool is still needed.

7. Create an ECS Fargate cluster

AWS Console → Elastic Container Service (ECS) → Clusters → Create cluster. Name: hello-world-cluster. Use the Fargate/serverless capacity option where shown. Create the cluster.

8. Register an ECS task definition

ECS → Task definitions → Create new task definition. Example for the Java Spring Boot service:

Setting	Example value
Task family	hello-world-svc-task
Launch type	AWS Fargate
Operating system / architecture	Linux / match your image
CPU / memory	0.5 vCPU / 1 GB
Container name	hello-world-svc
Image	123456789012.dkr.ecr.us-east-2.amazonaws.com/hello-world-svc:v1
Container port	8090 / TCP
Execution role	Role with AmazonECSTaskExecutionRolePolicy

The ECS task execution role allows ECR image pulls and CloudWatch log delivery. Configure awslogs logging. Your application may require environment variables and secrets; use AWS Secrets Manager for database passwords and JWT secrets.

```
SPRING_DATASOURCE_URL=jdbc:postgresql://<RDS_ENDPOINT>:5432/helloworld_wealth
SPRING_DATASOURCE_USERNAME=<DB_USER>
SPRING_DATASOURCE_PASSWORD=<FROM_SECRETS_MANAGER>
JWT_SHARED_SECRET=<FROM_SECRETS_MANAGER>
```

The RDS database must already exist and permit network connections from the ECS task security group. The SQL schema should be initialized once, not reset during every deployment.

9. Create the ECS service

ECS → Clusters → hello-world-cluster → Services → Create. Choose the hello-world-svc-task definition, Fargate, desired task count 1, VPC, subnets, and security group. For a production-style lab, use private subnets and an Application Load Balancer (ALB) to expose HTTP APIs; configure an ALB target group with target type IP and the correct application port.

For a short isolated demonstration only, a public subnet with assigned public IP and tightly restricted inbound port 8090 can be used. Do not leave an unrestricted security-group rule open.

10. Verify the running container

ECS → Clusters → hello-world-cluster → Services → hello-world-svc → Tasks. Look for RUNNING and desired count 1. If it fails, inspect service Events, task Stopped reason, exit code, and CloudWatch logs.

```
aws ecs list-clusters --region us-east-2
aws ecs list-services --cluster hello-world-cluster --region us-east-2
aws ecs describe-services --cluster hello-world-cluster --services hello-world-svc --region us-east-2
```

A RUNNING task does not guarantee the application is healthy. Test a real application endpoint through the ALB DNS name (or the task public IP only for a controlled lab), and verify RDS/Kafka connectivity as needed.

11. Deploy the other application services

Backend ECR repository	Application	Container port
hello-world-svc	Spring Boot API	8090
hello-world-holdings	Holdings service	6200
hello-world-rpt-svc	Python reporting API	7200
hello-world-auth	NestJS authentication	3000

Create an ECR repository, task definition, and ECS service for each backend. The two Angular frontends are static web builds best served from S3 + CloudFront rather than deployed as Fargate services. An ALB routes API traffic to the appropriate ECS target groups. RDS PostgreSQL and Kafka on EC2 are supporting services, not ECR images by themselves.

12. Troubleshooting checklist

Symptom	Check
AccessDenied when creating ECR	IAM policy or organization permission boundaries
Docker login fails	Correct account/region, active credentials, ECR permissions
Docker push denied	Repository URI, repository existence, push permissions
ECS cannot pull image	Task execution role, ECR URI/tag, network access
Task stops immediately	CloudWatch logs, startup variables, image architecture
API not reachable	ALB listener/target group, health check, SG, port
Database connection fails	RDS endpoint, secret, subnet routing, SG rules

13. Student lab success criteria

- Identify the correct AWS account using aws sts get-caller-identity.
- Create or locate an ECR private repository in the chosen region.
- Push one Docker image and verify its tag in ECR.
- Register an ECS task definition referencing that image.
- Create a Fargate service with one running task.
- Inspect logs and test an actual application endpoint.

Official AWS documentation

- AWS CLI installation: https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html
- IAM Identity Center CLI setup: https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-sso.html
- Push an image to ECR: https://docs.aws.amazon.com/AmazonECR/latest/userguide/docker-push-ecr-image.html
- ECS Fargate: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html

- AWS CloudShell: https://docs.aws.amazon.com/cloudshell/latest/userguide/welcome.html